Hybrid CNN and Binary Dung Beetle Optimizer Feature Selection for Multi-Class Plant Leaf Disease Classification

An IDP Project Report Submitted by

Shaik Sarfirose (Reg. No. 241FA18020)
Abdul Ajeem (Reg. No. 251LA18002)
Ravipati Raja Sri Lakshmif (Reg. No. 241FA18144)
Dhanalakota Namratha (Reg. No. 241FA18364)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Artificial Intelligence & Machine Learning

Under the Guidance of

Dr. B. Sudha Rani



Department of Advanced Computer Science and Engineering
School of Computing and Informatics
Vignan's Foundation for Science, Technology & Research
(Deemed to be University)
Vadlamudi, Guntur, Andhra Pradesh – 522213, India

November 2026

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

Department of Advanced Computer Science and Engineering

School of Computing and Informatics

Vignan's Foundation for Science, Technology & Research

(Deemed to be University)

Andhra Pradesh, India – 522213

CERTIFICATE

This is to certify that the project report entitled **"Hybrid CNN and Binary Dung Beetle Optimizer Feature Selection for Multi-Class Plant Leaf Disease Classification"**, being submitted by **Shaik Sarfirose (241FA18020), Abdul Ajeem (251LA18002), Ravipati Raja Sri Lakshmif (241FA18144), and Dhanalakota Namratha (241FA18364)**, in partial fulfilment of Bachelor of Technology in **Artificial Intelligence & Machine Learning**, Department of Advanced Computer Science and Engineering, Vignan's Foundation For Science Technology and Research (Deemed to be University), Vadlamudi, Guntur District, Andhra Pradesh, India, is a bonafide work carried out by them under my guidance and supervision.

Guide	**Project Co-ordinator**	**HOD, ACSE**
Dr. B. Sudha Rani	*[Coordinator Name]*	*[HoD Name]*

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

Department of Advanced Computer Science and Engineering

School of Computing and Informatics

Vignan's Foundation for Science, Technology & Research

(Deemed to be University)

Andhra Pradesh, India – 522213

DECLARATION

We hereby declare that our project work described in the project titled **"Hybrid CNN and Binary Dung Beetle Optimizer Feature Selection for Multi-Class Plant Leaf Disease Classification"** which is being submitted by us for the partial fulfilment in the department of ACSE, Vignan's Foundation for Science, Technology and Research (Deemed to be University), Vadlamudi, Guntur, Andhra Pradesh, and the result of investigations are carried out by us under the guidance of **Dr. B. Sudha Rani**.

Shaik Sarfirose
(241FA18020)

Abdul Ajeem
(251LA18002)

Ravipati Raja Sri Lakshmif
(241FA18144)

Dhanalakota Namratha
(241FA18364)

ACKNOWLEDGMENTS

First and foremost, we praise and thank **ALMIGHTY GOD** whose blessings have bestowed in me the will power and confidence to carry out my project.

We are grateful to our beloved founder and chairman **Dr. Lavu Rathaiah**, for giving us this opportunity to pursue our B.Tech in Vignan's Foundation for Science, Technology and Research.

We extend our thanks to our respected Vice Chancellor **Dr. P. Nagabhushan**, and our Registrar **Dr. P M V Rao**, for giving us this opportunity to do the project.

We extend our thanks to **Dr. Venkatesulu Dondeti**, Addl. Dean and Professor, Department of Advanced Computer Science and Engineering for his encouragement and guidance.

We extend our thanks to **Dr. D. Radha Rani**, Head of the Department of Advanced Computer Science and Engineering, for her support and leadership.

We feel it a pleasure to be indebted to our guide, **Dr. B. Sudha Rani**, Assistant Professor, Department of Advanced Computer Science and Engineering, for invaluable support, advice and encouragement.

We would like to thank [**Coordinator Name**] Project Co-ordinator, Department of Advanced Computer Science and Engineering for his support.

We also thank all the staff members of our department for extending their helping hands to make this project a success. We would also like to thank all my friends and my parents who have prayed and helped me during the project work.

ABSTRACT

Early and accurate identification of plant leaf diseases supports timely crop protection, but manual inspection is slow and requires expertise. This report presents a hybrid pipeline for 38-class leaf-disease classification in which a regularised convolutional neural network (CNN) is used as a feature extractor and a Binary Dung Beetle Optimizer (BDBO) selects a compact subset of the 256-dimensional global-average-pooled feature vector. The CNN was trained on 56,236 images of the New Plant Diseases Dataset (128×128 RGB), validated on 14,059 images, and tested on the 17,572 images of the dataset's separate `valid` folder. Selection was performed by a wrapper search that minimises a weighted sum of k-nearest-neighbour validation error and the fraction of features retained; Binary Particle Swarm Optimization (BPSO) was run as a comparison. A small dense classifier was then retrained on each feature subset. The end-to-end CNN reached 89.81% test accuracy. On the frozen features, a dense head using all 256 features reached 98.66%, a head using 120 BPSO-selected features reached 98.68%, and a head using 107 BDBO-selected features reached 98.47% (precision 98.48%, recall 98.47%, F1-score 98.47%). BDBO therefore removed 58.20% of the features at a cost of 0.19 percentage points relative to the full-feature head; it did not improve accuracy over that like-for-like baseline, and BPSO reached a slightly lower search fitness than BDBO in the single run reported. The large gap between the end-to-end CNN and the retrained heads is attributed to the retrained classifier, not to feature selection. All results come from one random seed, one dataset, and a held-out folder that may share source images with the training folder; the report therefore lists multi-seed repetition, significance testing, and field-image validation as required follow-up work.

Keywords: Plant disease classification, convolutional neural network, feature selection, dung beetle optimizer, particle swarm optimization, wrapper method, deep features.

Contents

Certificate

Declaration

Acknowledgments

Abstract

List of Figures

List of Tables

List of Acronyms/Abbreviations

List of Symbols

1 Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Problem Statement 2

1.4 Objectives 2

1.5 Scope of the Project 3

1.6 Organisation of the Report 3

2 Literature Review **4**

2.1 Deep Learning for Plant-Disease Recognition 4

2.2 Regularisation and Architectural Components 4

2.3 Feature Selection and Wrapper Methods 5

2.4 Swarm Optimizers and Binary Variants 5

2.5 Research Gap and Design Motivation 5

2.6 Summary 6

3 System Design and Methodology **7**

3.1 System Overview 7

3.2 Dataset 8

3.3 Preprocessing and Data Augmentation . . . 9
3.4 CNN Architecture . . . 9
3.5 CNN Training Procedure . . . 10
3.6 Feature Extraction . . . 10
3.7 Wrapper Feature Selection . . . 10
3.7.1 Problem formulation . . . 10
3.7.2 Binarisation . . . 11
3.7.3 Binary Dung Beetle Optimizer . . . 11
3.7.4 Binary PSO baseline . . . 12
3.8 Classifier Heads . . . 12
3.9 Evaluation Protocol . . . 13
3.10 Single-Image Inference . . . 13
3.11 Summary . . . 13

4 Implementation **14**
4.1 Software and Hardware Environment . . . 14
4.2 Data Pipeline . . . 14
4.3 CNN Implementation . . . 14
4.4 Feature Extractor . . . 15
4.5 Optimizer Implementation . . . 15
4.6 Classifier Heads . . . 15
4.7 Inference Module . . . 16
4.8 Saved Artifacts . . . 17
4.9 Summary . . . 17

5 Results and Discussion **18**
5.1 CNN Training Behaviour . . . 18
5.2 End-to-End CNN Evaluation . . . 18
5.3 Feature Selection Search . . . 19
5.4 Comparison of Models . . . 19
5.5 Interpreting the Comparison . . . 21
5.6 Per-Class Analysis of the BDBO Model . . . 21
5.7 Summary . . . 24

6 Limitations, Threats to Validity, and Further Experiments **25**
6.1 Single Run and Absence of Significance Tests . . . 25
6.2 Surrogate Fitness . . . 25
6.3 Validation Data Reuse . . . 25

6.4 Dataset Characteristics 26
6.5 End-to-End CNN Behaviour 26
6.6 Simplified Optimizer 26
6.7 Efficiency Claims 26
6.8 Code Reproducibility Issues 27
6.9 Ethical and Practical Considerations 27
6.10 Recommended Further Experiments 27
6.11 Summary 28

7 Conclusions and Future Directions **29**
7.1 Conclusions 29
7.2 Scope for Future Study 29

References **32**

List of Figures

3.1 Overall pipeline: CNN feature extraction, wrapper feature selection, and retrained classifier head. The binary mask **m** selects which of the 256 features are passed to the head. 7
3.2 Number of training images per class in the `train` folder (minimum 1,642; maximum 2,022). 8

4.1 Single-image predictions from the notebook. These are illustrative examples, not performance measurements. 16

5.1 Accuracy and loss of the end-to-end CNN over 20 epochs. Validation accuracy and loss fluctuate strongly while the training curves are smooth. 18
5.2 Confusion matrix of the end-to-end CNN on the test set. Off-diagonal mass is concentrated in a few columns, indicating that some classes act as "sinks" for misclassified images. 20
5.3 Best fitness (lower is better) versus iteration for BDBO and BPSO. BPSO ends at a slightly lower fitness; BDBO ends with fewer features. 21
5.4 Confusion matrix of the CNN + BDBO head on the test set. Most mass lies on the diagonal. 23

List of Tables

3.1 Dataset split used in the experiments 8
3.2 CNN architecture (input $128 \times 128 \times 3$) 9

4.1 CNN training configuration 15
4.2 Optimizer and surrogate settings 16

5.1 Selected epochs from the CNN training log (training-mode metrics) . 19
5.2 Test-set comparison (17,572 images, single run) 20
5.3 Per-class test results: F1-score of the end-to-end CNN (two decimals, as printed by the notebook) and precision, recall, and F1-score of the CNN + BDBO head (four decimals) 22

List of Acronyms/Abbreviations

BDBO	Binary Dung Beetle Optimizer
BN	Batch Normalization
BPSO	Binary Particle Swarm Optimization
CNN	Convolutional Neural Network
DBO	Dung Beetle Optimizer
FN / FP	False Negative / False Positive
GAP	Global Average Pooling
GPU	Graphics Processing Unit
k-NN	k-Nearest Neighbours
L2	Squared-norm (ridge) weight regularisation
PSO	Particle Swarm Optimization
ReLU	Rectified Linear Unit
TP / TN	True Positive / True Negative

List of Symbols

D	Dimension of the CNN feature vector ($D = 256$)
$\mathbf{f}$	CNN feature vector for one image, $\mathbf{f} \in \mathbb{R}^D$
$\mathbf{m}$	Binary feature mask, $\mathbf{m} \in \{0,1\}^D$
$\lvert\mathbf{m}\rvert$	Number of selected features (number of ones in $\mathbf{m}$)
$\mathbf{x}_i$	Continuous position of the i-th search agent in $[-1,1]^D$
$\mathbf{x}_b,\ \mathbf{x}_w$	Best and worst positions in the current population
N	Population size ($N = 20$)
$T_{\max}$	Number of optimizer iterations ($T_{\max} = 25$)
R	Shrinking boundary factor, $R = 1 - t/T_{\max}$
$\alpha,\ \beta$	Weights on error and feature ratio in the fitness function
$F(\mathbf{m})$	Wrapper fitness of mask $\mathbf{m}$ (lower is better)
$S(\cdot)$	Sigmoid transfer function
$w,\ c_1,\ c_2$	BPSO inertia weight and acceleration coefficients
$\odot$	Element-wise (Hadamard) product

Chapter 1

Introduction

This chapter explains the motivation for the project, states the problem addressed, lists the objectives, defines the scope, and outlines the organisation of the report.

1.1 Background

Plant diseases reduce crop yield and quality, and delays in recognising them allow infections to spread. Diagnosis in the field traditionally relies on visual inspection by trained personnel, which is slow, subjective, and unavailable to many growers. Image-based classification offers a way to automate part of this process: a photograph of a leaf is assigned to a healthy or diseased category, and the predicted category can guide further inspection or treatment.

Convolutional neural networks (CNNs) are the dominant approach to this task because they learn discriminative visual patterns such as lesion texture, colour change, and shape directly from labelled images [1, 3]. A CNN typically ends in a vector of learned features that is passed to a classifier. That vector may contain redundant or weakly informative dimensions, which raises the question of whether a smaller subset of the features can support classification equally well.

1.2 Motivation

Feature selection is a long-established way to reduce dimensionality while retaining the information that matters for a prediction task [10]. Wrapper methods evaluate candidate subsets with a learning algorithm and are well suited to metaheuristic search. Swarm-intelligence optimizers such as particle swarm optimization [11,12] and the more recent Dung Beetle Optimizer [14] have been proposed for search problems of this kind.

This project asks a concrete question: given a trained leaf-disease CNN, can a Binary Dung Beetle Optimizer find a substantially smaller subset of its 256 pooled features that preserves classification accuracy, and how does it compare with a Binary Particle Swarm Optimization baseline? A second motivation is methodological. Re-

ports of hybrid "CNN + optimizer" systems sometimes compare the hybrid against a weaker baseline, which can overstate the benefit of the optimizer. This report therefore compares each selected subset against a baseline trained under identical conditions on the same frozen features.

1.3 Problem Statement

The problem is to classify a leaf image into one of 38 plant-and-condition classes and, within a CNN-based pipeline, to identify a reduced subset of the CNN's 256-dimensional feature vector that preserves test accuracy. The solution must (i) train and evaluate the CNN without using the test images for model selection, (ii) search for a feature subset using a clearly defined objective that balances accuracy and subset size, and (iii) report the effect of the selection against fair baselines.

1.4 Objectives

The objectives of the project are:

1. Build a regularised CNN for 38-class leaf-disease classification and control overfitting using augmentation, batch normalization, L2 regularisation, dropout, and learning-rate and early-stopping callbacks.

2. Extract the 256-dimensional global-average-pooled feature vector for every image from the trained CNN.

3. Implement a Binary Dung Beetle Optimizer (BDBO) wrapper that selects a feature subset by minimising a combined error and subset-size objective.

4. Implement a Binary Particle Swarm Optimization (BPSO) wrapper as a comparison method.

5. Train identical dense classifier heads on all features, on the BPSO subset, and on the BDBO subset, and compare them on a held-out test set.

6. Report accuracy, precision, recall, F1-score, per-class behaviour, and feature reduction, and state clearly what the evidence does and does not show.

1.5 Scope of the Project

The project covers the CNN, the feature-selection search, the retrained classifier heads, and single-image inference, all implemented in a Google Colab notebook with TensorFlow/Keras and scikit-learn. Experiments use one public dataset of 38 classes, images resized to 128×128 pixels, and a single random seed. The report does not cover mobile or web deployment, field-captured images, disease severity estimation, localisation of lesions, or comparison with large pretrained architectures; the notebook includes a commented-out MobileNetV2 transfer-learning cell, but it was not executed and no result from it is reported.

1.6 Organisation of the Report

The report has seven chapters. Chapter 1 introduces the project. Chapter 2 reviews related work on CNN-based plant-disease recognition, feature selection, and swarm optimizers. Chapter 3 describes the dataset, the CNN, the BDBO and BPSO wrappers, and the evaluation protocol. Chapter 4 documents the implementation and configuration. Chapter 5 presents and discusses the results. Chapter 6 discusses limitations, threats to validity, and further experiments needed. Chapter 7 concludes the report and lists future work.

Chapter 2

Literature Review

This chapter reviews the research themes behind the project: deep learning for plant-disease recognition, feature selection with wrapper methods, and swarm-based optimizers. The aim is to place the project in context and identify the gap it addresses.

2.1 Deep Learning for Plant-Disease Recognition

Hughes and Salathé introduced the PlantVillage collection of labelled leaf images as an open resource for computer-vision research on crop diseases [2]. Mohanty *et al.* trained deep CNNs on PlantVillage images covering 14 crop species and 26 diseases (38 classes) and reported accuracy above 99% on held-out images from the same collection, while noting that performance degrades when images are captured under conditions different from the training data [1]. Ferentinos trained several CNN architectures on a larger open database of leaf images and likewise reported accuracy above 99% [3]. Barbedo examined how dataset size and variety affect CNN and transfer-learning performance for plant-disease classification [4], and Too *et al.* compared the fine-tuning of several pretrained networks for plant-disease identification [5].

Two lessons from this literature shape the present work. First, high accuracy on laboratory-style benchmark images is common, so differences between methods are often small and need careful, like-for-like comparison. Second, reported accuracy depends on how the data are split and on whether the evaluation images are genuinely independent of the training images, which motivates the validity discussion in Chapter 6.

2.2 Regularisation and Architectural Components

The CNN in this project combines several standard components. Batch normalization stabilises and accelerates training by normalising intermediate activations [6]. Dropout reduces co-adaptation of units by randomly omitting them during training [7]. Global average pooling replaces a large fully connected layer with a spatial average per channel, which sharply reduces the number of parameters and the ten-

dency to overfit [8]. The Adam optimizer is used for gradient-based training [9]. These components are described in Chapter 3.

2.3 Feature Selection and Wrapper Methods

Feature selection removes irrelevant or redundant input dimensions to improve efficiency and, in some cases, generalisation [10]. Filter methods score features independently of any classifier, whereas wrapper methods evaluate candidate subsets by training or applying a classifier and are therefore closer to the final task but more expensive. Wrapper search over D features is a combinatorial problem with 2^D candidate subsets, which makes heuristic and metaheuristic search attractive. A common formulation minimises a weighted sum of classification error and the fraction of features retained, which is the formulation used in this project.

2.4 Swarm Optimizers and Binary Variants

Particle swarm optimization (PSO) updates a population of candidate solutions using each particle's own best position and the best position found by the swarm [11]. Kennedy and Eberhart proposed a discrete binary version in which velocities are mapped to bit probabilities [12]. Transfer functions that map continuous positions to binary decisions, such as S-shaped (sigmoid) and V-shaped functions, are widely used to binarise continuous metaheuristics [13].

The Dung Beetle Optimizer (DBO) was proposed by Xue and Shen as a population-based metaheuristic inspired by the rolling, dancing, foraging, stealing, and breeding behaviour of dung beetles [14]. In the version described in the paper, the population is divided into sub-groups that follow different update rules: some agents roll balls and explore the search space, some agents breed and some forage within a boundary around the best solution, and some agents steal from the best-known region. In this project the sub-group update rules are applied in a continuous space and the resulting positions are binarised with a sigmoid transfer function, as is usual for wrapper feature selection. The implementation is a simplified adaptation, described in Section 3.7.

2.5 Research Gap and Design Motivation

The literature shows that CNNs classify benchmark leaf images very accurately and that swarm optimizers can be used for wrapper feature selection. What is less often made explicit is the like-for-like effect of the selection step: a hybrid pipeline should

be compared with a classifier trained on the *same* frozen features with *all* dimensions retained, and with at least one alternative selection method. This project follows that design. It reports the end-to-end CNN, a full-feature head, a BPSO-selected head, and a BDBO-selected head on the same held-out test set, and it states the result as found, including where the optimizer does not improve accuracy.

2.6 Summary

Deep CNNs are well established for leaf-disease recognition, and wrapper feature selection with swarm optimizers is a recognised way to shrink feature vectors. The contribution of this report is a transparent, reproducible comparison of BDBO and BPSO selection on the pooled features of a regularised CNN, with the selection effect isolated from the effect of retraining the classifier.

Chapter 3

System Design and Methodology

This chapter describes the dataset, the CNN, the feature-extraction step, the BDBO and BPSO wrapper searches, the retrained classifier heads, and the evaluation protocol.

3.1 System Overview

The pipeline has four stages (Figure 3.1). First, a regularised CNN is trained end to end on leaf images. Second, its weights are frozen and the 256-dimensional output of the global-average-pooling layer is extracted for every image. Third, a wrapper search (BDBO, with BPSO as a comparison) finds a binary mask over the 256 features. Fourth, a small dense classifier is trained on the masked features and evaluated on the held-out test set.

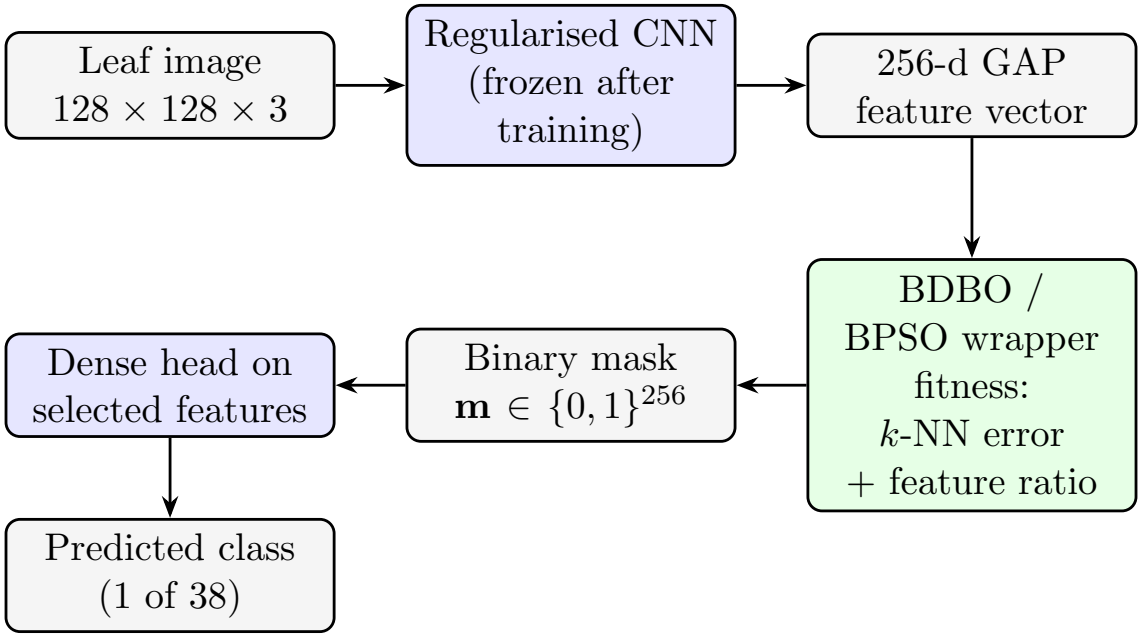


Figure 3.1: Overall pipeline: CNN feature extraction, wrapper feature selection, and retrained classifier head. The binary mask $\mathbf{m}$ selects which of the 256 features are passed to the head.

3.2 Dataset

Experiments use the New Plant Diseases Dataset obtained through the `kagglehub` package (identifier `vipoooool/new-plant-diseases-dataset`) [17]. The class labels follow the PlantVillage taxonomy of crop–condition pairs [2]: 38 classes spanning apple, blueberry, cherry, corn (maize), grape, orange, peach, bell pepper, potato, raspberry, soybean, squash, strawberry, and tomato, with both healthy and diseased categories.

The dataset provides a `train` folder and a `valid` folder. The `train` folder holds 70,295 images, which were divided 80%/20% with a fixed seed (42) into 56,236 training and 14,059 validation images. The `valid` folder holds 17,572 images and is used here as the held-out test set; the dataset does not ship a separate third folder. Class sizes in the `train` folder range from 1,642 to 2,022 images (Figure 3.2), so the data are close to balanced. Table 3.1 summarises the split.

Table 3.1: Dataset split used in the experiments

Subset	Images	Source
Training	56,236	80% of `train` folder (seed 42)
Validation	14,059	20% of `train` folder (seed 42)
Test	17,572	`valid` folder (held out)
Classes	38	–
Image size	$128 \times 128 \times 3$	resized

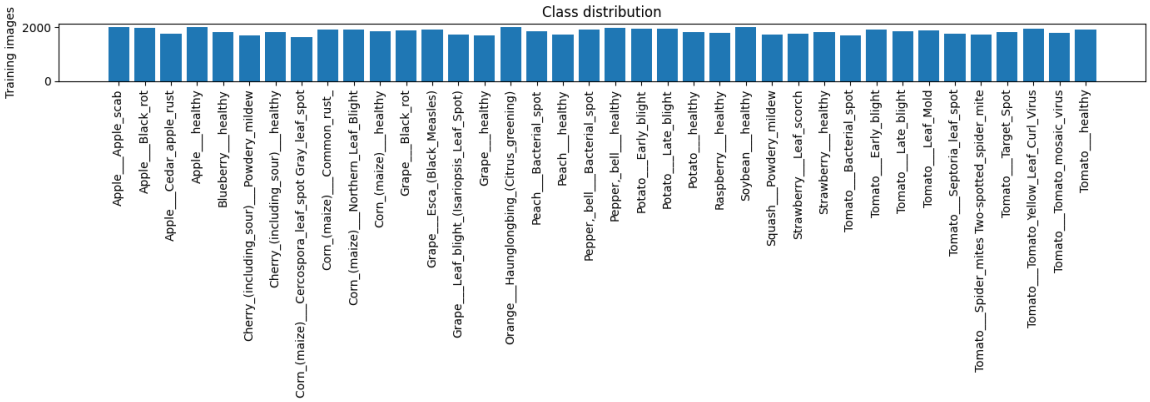


Figure 3.2: Number of training images per class in the `train` folder (minimum 1,642; maximum 2,022).

3.3 Preprocessing and Data Augmentation

All images are resized to 128×128 pixels and rescaled to $[0, 1]$ by dividing by 255. Augmentation is built into the model as layers that are active only during training: random horizontal flip, random rotation (factor 0.15), random zoom (0.15), random contrast (0.15), and random translation (0.1 in each direction). Validation and test images are not augmented.

3.4 CNN Architecture

The CNN has four convolutional blocks with 32, 64, 128, and 256 filters. Each block contains two 3×3 convolutions with "same" padding, each followed by batch normalization and a ReLU activation, then 2×2 max pooling and dropout with rate 0.25. All convolution and dense kernels use L2 regularisation with coefficient 5×10^{-4}. The final feature map is reduced by global average pooling to a 256-dimensional vector (the layer named `gap_features`), followed by a dense layer of 128 ReLU units, batch normalization, dropout with rate 0.5, and a 38-way softmax output. The model has 1,214,406 parameters (1,212,230 trainable and 2,176 non-trainable). Table 3.2 lists the layers.

Table 3.2: CNN architecture (input $128 \times 128 \times 3$)

Layer(s)	**Output shape**	**Parameters**
Augmentation (train only)	$128 \times 128 \times 3$	0
Block 1: 2×[Conv 32, BN, ReLU], MaxPool, Dropout	$64 \times 64 \times 32$	10,400
Block 2: 2×[Conv 64, BN, ReLU], MaxPool, Dropout	$32 \times 32 \times 64$	55,936
Block 3: 2×[Conv 128, BN, ReLU], MaxPool, Dropout	$16 \times 16 \times 128$	222,464
Block 4: 2×[Conv 256, BN, ReLU], MaxPool, Dropout	$8 \times 8 \times 256$	887,296
Global average pooling (`gap_features`)	256	0
Dense 128, ReLU; BN; Dropout 0.5	128	33,408
Dense 38, softmax	38	4,902
Total		**1,214,406**

3.5 CNN Training Procedure

The CNN is trained with the Adam optimizer [9] (learning rate 3×10^{-4}), categorical cross-entropy loss, batch size 64, and at most 30 epochs, using mixed-precision (float16) arithmetic on a GPU. Three callbacks control training: early stopping on validation loss (patience 7, restoring the best weights), learning-rate reduction on plateau (factor 0.3, patience 3, minimum 10^{-7}), and a checkpoint that saves the model with the best validation accuracy. The random seed is fixed to 42.

3.6 Feature Extraction

After training, the model is truncated at the `gap_features` layer to form a feature extractor. For each image i it returns $\mathbf{f}_i \in \mathbb{R}^D$ with $D = 256$. Features are computed once, without augmentation, for the training, validation, and test images and cast to 32-bit floats for use with scikit-learn.

3.7 Wrapper Feature Selection

3.7.1 Problem formulation

A feature subset is encoded as a binary mask $\mathbf{m} \in \{0,1\}^D$, where $m_j = 1$ keeps feature j. The quality of a mask is measured by the fitness (to be minimised)

$$F(\mathbf{m}) = \alpha \left(1 - \mathrm{Acc}_{k\text{-NN}}(\mathbf{m})\right) + \beta \frac{|\mathbf{m}|}{D}, \tag{3.7.1}$$

with $\alpha = 0.99$ and $\beta = 0.01$. $\mathrm{Acc}_{k\text{-NN}}(\mathbf{m})$ is the validation accuracy of a k-nearest-neighbour classifier ($k = 5$) fitted on a random subsample of 6,000 training feature vectors restricted to the selected columns and scored on the full validation feature set. A mask with no selected feature receives the worst fitness, $F = 1$. The k-NN classifier is a cheap surrogate that can be evaluated many times; the reported accuracies in Chapter 5 come from a trained dense head, not from k-NN.

3.7.2 Binarisation

Both optimizers work with real-valued positions $\mathbf{x} \in [-1, 1]^D$. A position is converted to a mask by the sigmoid transfer function,

$$S(x) = \frac{1}{1 + e^{-x}}, \qquad m_j = \begin{cases} 1, & S(x_j) > 0.5 \\ 0, & \text{otherwise,} \end{cases} \tag{3.7.2}$$

which is equivalent to keeping feature j when $x_j > 0$.

3.7.3 Binary Dung Beetle Optimizer

The population of $N = 20$ agents is initialised uniformly in $[-1, 1]^D$ and evolved for $T_{\max} = 25$ iterations. Agents are assigned to four roles by index: $\lfloor 0.2N \rfloor = 4$ ball-rolling beetles, $\lfloor 0.3N \rfloor = 6$ brood-ball (breeding) beetles, 6 foraging beetles, and the remaining 4 thief beetles. Let $\mathbf{x}_b$ and $\mathbf{x}_w$ be the best and worst positions at the start of an iteration, $R = 1 - t/T_{\max}$ the shrinking boundary factor, and $\mathbf{u}$, $\mathbf{b}_1$, $\mathbf{b}_2$ vectors with entries drawn uniformly from $[-1, 1]$, $\mathbf{c}_1, \mathbf{c}_2$ from $[0, 1]$, and $\mathbf{g}$ from $[0.5, 1.5]$. With boundaries $\mathbf{L}^* = \mathrm{clip}\big(\mathbf{x}_b(1 - R)\big)$ and $\mathbf{U}^* = \mathrm{clip}\big(\mathbf{x}_b(1 + R)\big)$, the update rules are:

$$\text{Rolling:} \quad \mathbf{x}_i \leftarrow \mathbf{x}_i + a\, k\, \mathbf{x}_i + b\, |\mathbf{x}_i - \mathbf{x}_w| \odot \mathbf{u}, \tag{3.7.3}$$

$$\text{Breeding:} \quad \mathbf{x}_i \leftarrow \mathbf{x}_b + \mathbf{b}_1 \odot (\mathbf{x}_i - \mathbf{L}^*) + \mathbf{b}_2 \odot (\mathbf{x}_i - \mathbf{U}^*), \tag{3.7.4}$$

$$\text{Foraging:} \quad \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{c}_1 \odot (\mathbf{x}_i - \mathbf{L}^*) + \mathbf{c}_2 \odot (\mathbf{x}_i - \mathbf{U}^*), \tag{3.7.5}$$

$$\text{Stealing:} \quad \mathbf{x}_i \leftarrow \mathbf{x}_b + s\, \mathbf{g} \odot \big(|\mathbf{x}_i - \mathbf{x}_b| + |\mathbf{x}_i - \mathbf{x}_w|\big), \tag{3.7.6}$$

where $a = +1$ with probability 0.9 and -1 otherwise, $k = 0.1$, $b = 0.3$, and $s = 0.5$. After every update the position is clipped to $[-1, 1]^D$. Rolling agents explore broadly, breeding and foraging agents search within a boundary that shrinks around the best solution as t increases, and thief agents search near the best solution. The global best mask is updated whenever an agent improves it. Algorithm 1 summarises the procedure.

This is a simplified adaptation of the original optimizer [14]: it applies the four role-specific updates above in a continuous space and binarises with Equation 3.7.2, and it does not reproduce every detail of the original algorithm (for example, the obstacle-encounter "dancing" step of the rolling phase and the use of the previous-iteration position). It should therefore be described as a BDBO-style optimizer, and results should not be attributed to the original algorithm without checking the details

against the source paper.

Algorithm 1 Binary Dung Beetle Optimizer for feature selection (as implemented)

Require: Feature matrices, population size N, iterations $T_{\max}$, dimension D
Ensure: Best binary mask $\mathbf{m}^*$
Initialise $\mathbf{x}_i \sim \mathcal{U}(-1,1)^D$ for $i = 1, \ldots, N$
Evaluate $F_i \leftarrow F(\text{binarise}(\mathbf{x}_i))$; set $\mathbf{x}_b$ to the best agent
for $t = 1$ **to** $T_{\max}$ **do**
    $\mathbf{x}_w \leftarrow$ worst agent; $R \leftarrow 1 - t/T_{\max}$
    for $i = 1$ **to** N **do**
        Update $\mathbf{x}_i$ with the rolling, breeding, foraging, or stealing rule according to its role
        Clip $\mathbf{x}_i$ to $[-1,1]^D$
    end for
    Re-evaluate all fitness values; update $\mathbf{x}_b$ if an agent improves on the best fitness
end for
return binarise($\mathbf{x}_b$)

3.7.4 Binary PSO baseline

BPSO uses the same population size, iteration count, binarisation, and fitness function. Velocities are initialised in $[-0.1, 0.1]$ and updated as

$$\mathbf{v}_i \leftarrow w\,\mathbf{v}_i + c_1\,\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i) + c_2\,\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i), \qquad \mathbf{x}_i \leftarrow \text{clip}(\mathbf{x}_i + \mathbf{v}_i), \tag{3.7.7}$$

with $w = 0.7$, $c_1 = c_2 = 1.5$, personal bests $\mathbf{p}_i$, global best $\mathbf{g}$, and random vectors $\mathbf{r}_1, \mathbf{r}_2 \sim \mathcal{U}(0,1)^D$. The two optimizers use different random-number streams (seeds 42 and 43), so their initial populations differ.

3.8 Classifier Heads

Three identical dense heads are trained on frozen features: one on all 256 features (the full-feature baseline), one on the BPSO subset, and one on the BDBO subset. Each head is Dense(128, ReLU, L2 10^{-4}) $\rightarrow$ Dropout(0.4) $\rightarrow$ Dense(38, softmax). Training uses Adam (learning rate 10^{-3}), sparse categorical cross-entropy, batch size 64, at most 30 epochs, and early stopping on validation loss (patience 5, best weights restored). The head training and validation sets come from a stratified 80%/20% split (seed 42) of the training-folder features; the test features are used only for the final evaluation.

3.9 Evaluation Protocol

Test performance is reported as accuracy, precision, recall, and F1-score. For a class c,

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}, \tag{3.9.1}$$

and the weighted averages use class supports as weights. Feature reduction is

$$\text{Reduction} = \left(1 - \frac{|\mathbf{m}|}{D}\right) \times 100\%. \tag{3.9.2}$$

Per-class precision, recall, F1-score, and a confusion matrix are also reported.

3.10 Single-Image Inference

For a new image, the pipeline resizes it to 128×128, divides by 255, passes it through the feature extractor, keeps the columns indicated by the saved mask, and applies the trained head. It returns the predicted class and the softmax confidence.

3.11 Summary

The methodology separates feature learning (CNN), feature selection (BDBO or BPSO with a k-NN surrogate fitness), and classification (a dense head retrained on each subset). The design allows the effect of selecting features to be compared with training on all features under identical settings.

Chapter 4

Implementation

This chapter records how the methodology was implemented, the software environment, and the configuration values used in the reported run. All values are taken from the project notebook and its recorded outputs.

4.1 Software and Hardware Environment

The experiments were run in Google Colab on a single NVIDIA Tesla T4 GPU (15,360 MiB memory) with TensorFlow 2.21.0 [15] and Keras for the CNN, scikit-learn [16] for the k-NN surrogate, metrics, and data splitting, NumPy for the optimizers, and Matplotlib and Seaborn for plots. GPU memory growth was enabled and the `mixed_float16` policy was used, with the final softmax layer kept in 32-bit precision for numerical stability. The dataset was downloaded with `kagglehub`, and the folder containing `train` and `valid` was located automatically.

4.2 Data Pipeline

Images are read with `image_dataset_from_directory` with categorical labels, 128×128 resizing, and batch size 64. The training and validation subsets are produced from the same `train` folder with `validation_split`=0.2 and seed 42, and the `valid` folder is loaded separately as the test set with the same class-name order. A `Rescaling(1/255)` layer is mapped over all three datasets. The training set is cached, shuffled (buffer 1,000), and prefetched.

4.3 CNN Implementation

The CNN is built with the Keras functional API through a reusable `conv_block` function (two Conv–BN–ReLU units, max pooling, dropout). The data-augmentation `Sequential` model is the first layer, so augmentation is applied only when the model is called in training mode. Table 4.1 lists the training configuration.

Table 4.1: CNN training configuration

Setting	Value
Input size / batch size	$128 \times 128 \times 3$ / 64
Optimizer / learning rate	Adam / 3×10^{-4}
Loss	Categorical cross-entropy
Maximum epochs	30
Regularisation	L2 (5×10^{-4}), dropout (0.25 in blocks, 0.5 in head), batch normalization
Augmentation	Flip (horizontal), rotation 0.15, zoom 0.15, contrast 0.15, translation 0.1
Early stopping	Monitor `val_loss`, patience 7, restore best weights
LR schedule	ReduceLROnPlateau: factor 0.3, patience 3, minimum 10^{-7}
Checkpoint	Best `val_accuracy`
Precision / seed	Mixed float16 / 42

4.4 Feature Extractor

A second Keras model is created whose input is the CNN input and whose output is the `gap_features` layer. In the later pipeline cell, the 256-dimensional layer is located programmatically by scanning for a two-dimensional output of width 256, and its output shape is checked against the sizes of the saved masks before any head is trained.

4.5 Optimizer Implementation

The wrapper fitness (Equation 3.7.1), BDBO, and BPSO are implemented in NumPy. Table 4.2 gives the settings. Each fitness evaluation fits a k-NN model on a 6,000-sample subset of the training features, which keeps the 20×26 evaluations per optimizer run tractable on CPU.

4.6 Classifier Heads

A `train_classifier` function builds the dense head for a given mask (or all features if the mask is `None`), trains it with early stopping, evaluates it on the test features, and returns the model, the selected indices, and a result record containing the number of features and the test accuracy. The same function is called for the full-feature baseline, the BPSO mask, and the BDBO mask, which guarantees identical training settings. The trained BDBO head, the binary mask, and the selected

Table 4.2: Optimizer and surrogate settings

Setting	BDBO	BPSO
Population / iterations	20 / 25	20 / 25
Search space	$[-1, 1]^{256}$	$[-1, 1]^{256}$
Binarisation	Sigmoid, threshold 0.5	Sigmoid, threshold 0.5
Role split / parameters	4 / 6 / 6 / 4; k=0.1, b=0.3, s=0.5	w=0.7, c_1=c_2=1.5
Random seed	42	43
Fitness weights	α=0.99, β=0.01	α=0.99, β=0.01
Surrogate classifier	k-NN, k=5	k-NN, k=5
Surrogate training subset	6,000 samples	6,000 samples

indices are saved as `CNN_DBO_final_classifier.keras`, `DBO_feature_mask.npy`, and `DBO_feature_indices.npy`.

4.7 Inference Module

Two inference routines were implemented and exercised on uploaded images. The first loads the end-to-end CNN and predicts directly. The second loads the CNN, the saved head, the mask, and the class-name list, extracts features, applies the mask, and predicts. Both resize to 128 × 128 and divide pixel values by 255, matching the training pipeline. Figure 4.1 shows one example from each routine.



(a) End-to-end CNN: predicted Tomato Yellow Leaf Curl Virus, confidence 95.72%.



(b) CNN + BDBO: predicted Apple scab, confidence displayed as 100.00%.

Figure 4.1: Single-image predictions from the notebook. These are illustrative examples, not performance measurements.

4.8 Saved Artifacts

The notebook saves the best CNN checkpoint (`best_leaf_disease_model.keras`), the final CNN (`trained_leaf_disease_model.keras`), the class names (`class_names.json`), the training history (`training_hist.json`), and the BDBO artifacts listed above. These files are sufficient to reproduce inference without retraining.

4.9 Summary

The implementation uses standard TensorFlow/Keras and scikit-learn components plus two short NumPy optimizers. Using one function to train every head keeps the comparison controlled.

Chapter 5

Results and Discussion

This chapter reports the results of the experiments described in Chapters 3 and 4, all of which come from a single run with seed 42, and discusses what they show. Test results use the 17,572 images of the held-out `valid` folder.

5.1 CNN Training Behaviour

Training ran for 20 epochs before early stopping, and the weights of epoch 13 were restored. Epochs took about 130 s on the T4 GPU after a slower first epoch (285 s). Training accuracy rose steadily from 52.1% to 98.6% in training mode, whereas validation accuracy was highly variable, ranging from 22.6% in the first epoch to a peak of 89.2% at epoch 13 (Figure 5.1, Table 5.1). Validation loss was lowest at epoch 13 (0.561), which is why those weights were restored. The learning rate was reduced several times by the plateau callback.

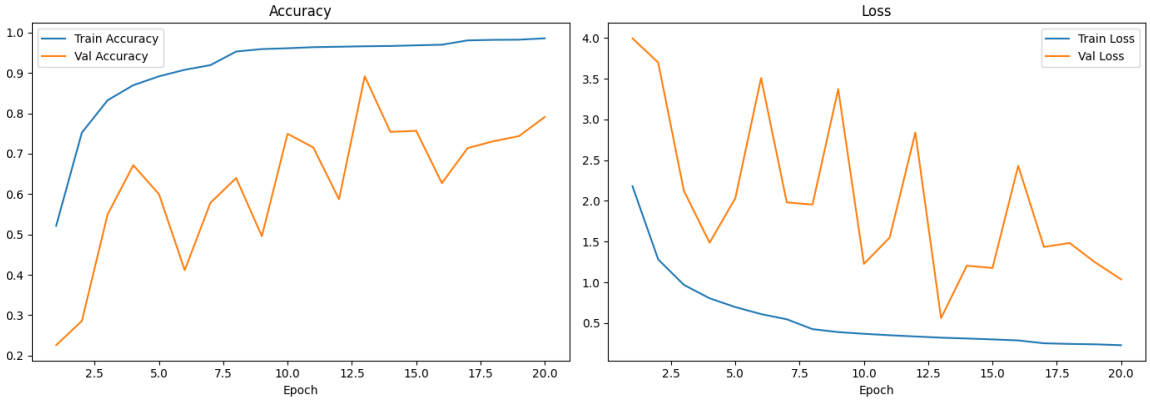


Figure 5.1: Accuracy and loss of the end-to-end CNN over 20 epochs. Validation accuracy and loss fluctuate strongly while the training curves are smooth.

5.2 End-to-End CNN Evaluation

With the restored weights, evaluation-mode accuracy was 90.35% on the training set, 89.16% on the validation set, and 89.81% on the test set; the train–test gap was

Table 5.1: Selected epochs from the CNN training log (training-mode metrics)

Epoch	Train acc.	Train loss	Val acc.	Val loss
1	0.5212	2.1800	0.2257	3.9949
5	0.8917	0.6971	0.5995	2.0343
10	0.9611	0.3694	0.7493	1.2275
13 (restored)	0.9660	0.3220	0.8916	0.5614
20 (stopped)	0.9857	0.2296	0.7909	1.0374

0.55 percentage points, so the usual symptom of overfitting (a large train–test gap) is absent. The weighted-average precision, recall, and F1-score on the test set were 0.92, 0.90, and 0.90. Performance was uneven across classes (Figure 5.2): for example Soybean healthy had recall 1.00 but precision 0.47, meaning many images of other classes were assigned to it, while Bell Pepper healthy had precision 1.00 but recall 0.51. Several other classes (Peach healthy, Tomato Target Spot, Apple healthy) had F1-scores of 0.78 or below.

5.3 Feature Selection Search

Both optimizers lowered the wrapper fitness over 25 iterations (Figure 5.3). BDBO started from a best fitness of 0.0334 (110 features) and ended at 0.0318 with 107 features. BPSO started from 0.0352 (112 features) and ended at 0.0310 with 120 features. The final fitness values correspond, through Equation 3.7.1, to a k-NN validation accuracy of approximately 97.2% for the BDBO subset and 97.3% for the BPSO subset. BPSO therefore found the better objective value in this run, using 13 more features, and BDBO found the smaller subset. Both searches converged to roughly 40–47% of the original dimensionality, which suggests that a large share of the 256 features is not needed by the surrogate classifier.

5.4 Comparison of Models

Table 5.2 compares the models on the test set. The three dense heads were trained with identical settings on frozen features and differ only in the features they receive. The head parameter counts follow from the layer sizes in Section 3.8.

The BDBO head classified 33 fewer test images correctly than the full-feature head and 37 fewer than the BPSO head. These differences are 0.19 and 0.21 percentage points. For the BDBO head the weighted precision, recall, and F1-score were 0.9848, 0.9847, and 0.9847, and the macro averages were 0.9846, 0.9847, and 0.9846.

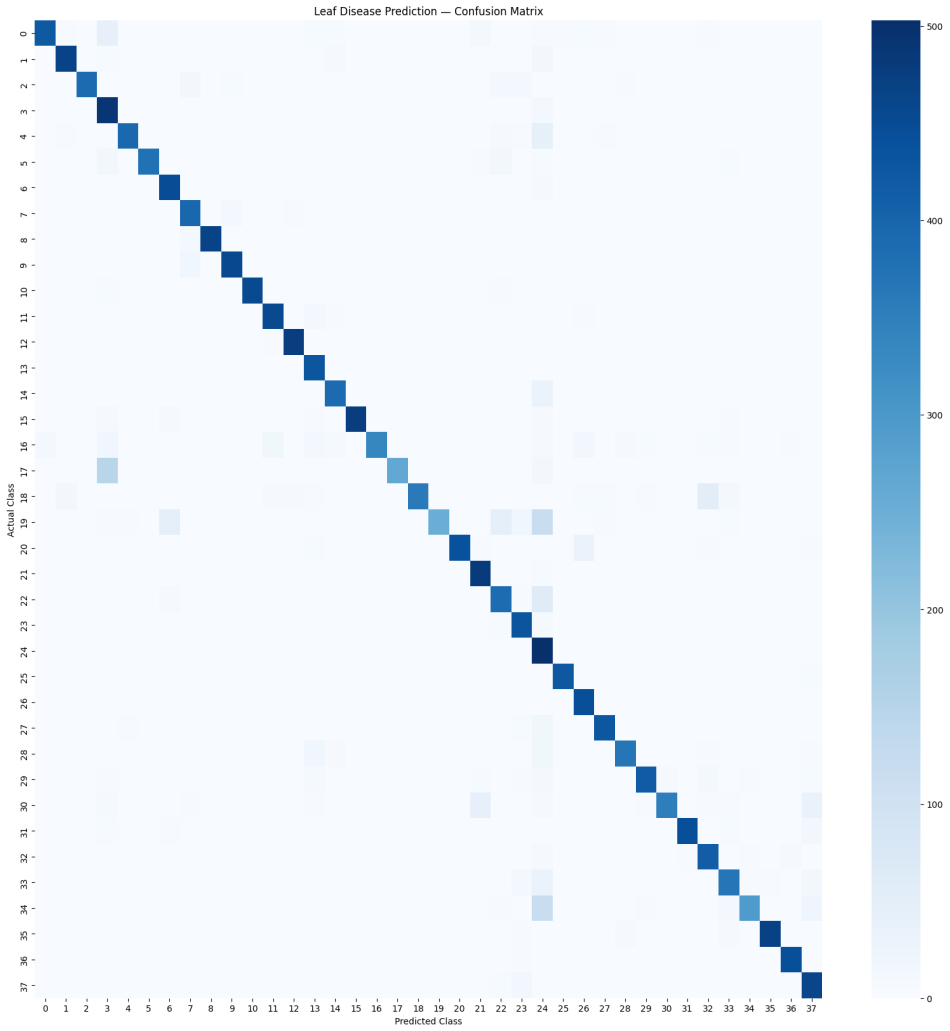


Figure 5.2: Confusion matrix of the end-to-end CNN on the test set. Off-diagonal mass is concentrated in a few columns, indicating that some classes act as "sinks" for misclassified images.

Table 5.2: Test-set comparison (17,572 images, single run)

Model	Features	Reduction	Test acc.	Correct	Head params
End-to-end CNN	256	0%	89.81%	–	–
Head, all features	256	0%	98.66%	17,336	37,798
Head + BPSO	120	53.13%	98.68%	17,340	20,390
Head + BDBO	107	58.20%	98.47%	17,303	18,726

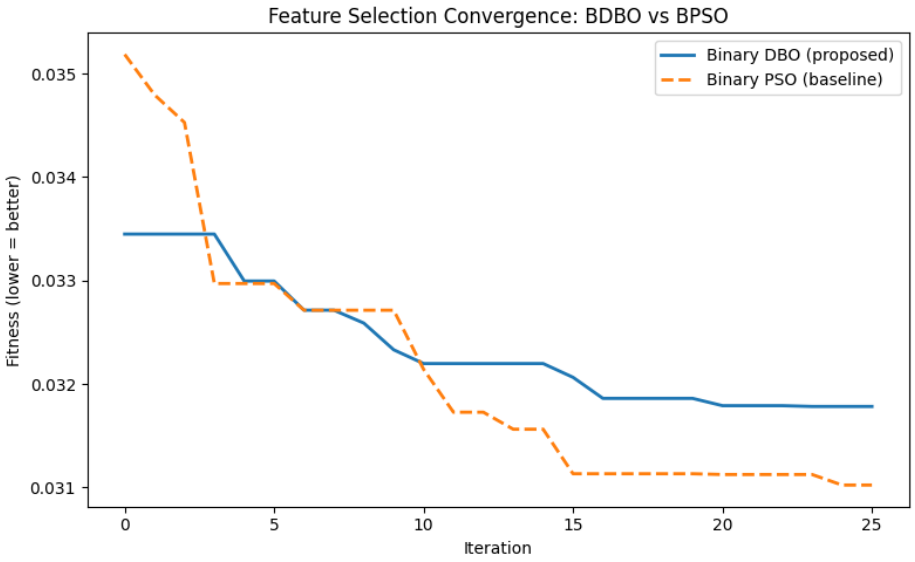


Figure 5.3: Best fitness (lower is better) versus iteration for BDBO and BPSO. BPSO ends at a slightly lower fitness; BDBO ends with fewer features.

5.5 Interpreting the Comparison

A comparison between the end-to-end CNN (89.81%) and the BDBO head (98.47%) would show an improvement of 8.66 percentage points. That improvement should not be attributed to BDBO. The full-feature head, which uses no selection, reaches 98.66%, and the BPSO head reaches 98.68%. The jump therefore comes from replacing the end-to-end classifier by a head retrained on the extracted features, not from removing features. The end-to-end network's lower test accuracy is consistent with its erratic validation behaviour (Section 5.1), and the cause was not isolated.

What the results do support is that a subset of 107 of the 256 features retains almost all of the accuracy of the full set: the accuracy cost of removing 58.20% of the features was 0.19 percentage points in this run. Whether that cost is acceptable depends on the application, and the benefit is limited because the convolutional backbone still has to be evaluated in full; only the head is smaller (18,726 versus 37,798 parameters, a 50.5% reduction).

5.6 Per-Class Analysis of the BDBO Model

Table 5.3 gives per-class results for the BDBO head alongside the F1-scores of the end-to-end CNN. The BDBO head achieved F1-scores of at least 0.95 for every class. The lowest were Tomato Target Spot (0.9481), Tomato Spider Mites (0.9574), and Corn Cercospora/Gray Leaf Spot (0.9586). These are diseases whose symptoms are

small spots or discolourations that resemble those of other conditions on the same or similar crops, which is a plausible explanation, although the notebook does not include an error analysis that confirms it. Classes that were poorly handled by the end-to-end CNN (Soybean healthy, Bell Pepper healthy, Peach healthy) are handled well by the retrained head, with F1-scores of 0.9671 or higher.

Table 5.3: Per-class test results: F1-score of the end-to-end CNN (two decimals, as printed by the notebook) and precision, recall, and F1-score of the CNN + BDBO head (four decimals)

Class	**CNN F1**	**P**	**R**	**F1**	**Support**
Apple: Apple scab	0.90	0.9805	0.9960	0.9882	504
Apple: Black rot	0.95	0.9960	0.9960	0.9960	497
Apple: Cedar apple rust	0.94	0.9799	0.9977	0.9887	440
Apple: healthy	0.78	0.9879	0.9781	0.9830	502
Blueberry: healthy	0.92	0.9740	0.9912	0.9825	454
Cherry: Powdery mildew	0.94	0.9928	0.9857	0.9893	421
Cherry: healthy	0.93	0.9890	0.9868	0.9879	456
Corn: Cercospora/Gray leaf spot	0.93	0.9563	0.9610	0.9586	410
Corn: Common rust	0.99	0.9937	0.9937	0.9937	477
Corn: Northern Leaf Blight	0.96	0.9807	0.9602	0.9703	477
Corn: healthy	0.99	1.0000	0.9978	0.9989	465
Grape: Black rot	0.95	0.9936	0.9936	0.9936	472
Grape: Esca (Black measles)	0.98	0.9958	0.9938	0.9948	480
Grape: Leaf blight (Isariopsis)	0.93	0.9977	1.0000	0.9988	430
Grape: healthy	0.92	1.0000	0.9976	0.9988	423
Orange: Huanglongbing (greening)	0.97	0.9960	0.9960	0.9960	503
Peach: Bacterial spot	0.85	0.9977	0.9651	0.9812	459
Peach: healthy	0.76	0.9931	0.9954	0.9942	432
Pepper, bell: Bacterial spot	0.86	0.9774	0.9958	0.9865	478
Pepper, bell: healthy	0.67	0.9874	0.9477	0.9671	497
Potato: Early blight	0.95	0.9979	0.9938	0.9959	485
Potato: Late blight	0.93	0.9753	0.9753	0.9753	485
Potato: healthy	0.82	0.9697	0.9825	0.9760	456
Raspberry: healthy	0.91	0.9779	0.9955	0.9866	445
Soybean: healthy	0.64	0.9822	0.9842	0.9832	505
Squash: Powdery mildew	0.98	0.9909	1.0000	0.9954	434
Strawberry: Leaf scorch	0.93	0.9910	0.9932	0.9921	444

Class	CNN F1	P	R	F1	Support
Strawberry: healthy	0.95	0.9913	0.9978	0.9945	456
Tomato: Bacterial spot	0.91	0.9905	0.9765	0.9834	425
Tomato: Early blight	0.92	0.9687	0.9667	0.9677	480
Tomato: Late blight	0.85	0.9714	0.9546	0.9630	463
Tomato: Leaf Mold	0.97	0.9979	0.9915	0.9947	470
Tomato: Septoria leaf spot	0.89	0.9662	0.9839	0.9750	436
Tomato: Spider mites	0.86	0.9585	0.9563	0.9574	435
Tomato: Target Spot	0.77	0.9379	0.9584	0.9481	457
Tomato: Yellow Leaf Curl Virus	0.98	0.9898	0.9857	0.9877	490
Tomato: Tomato mosaic virus	0.98	0.9955	0.9955	0.9955	448
Tomato: healthy	0.88	0.9938	0.9979	0.9959	481

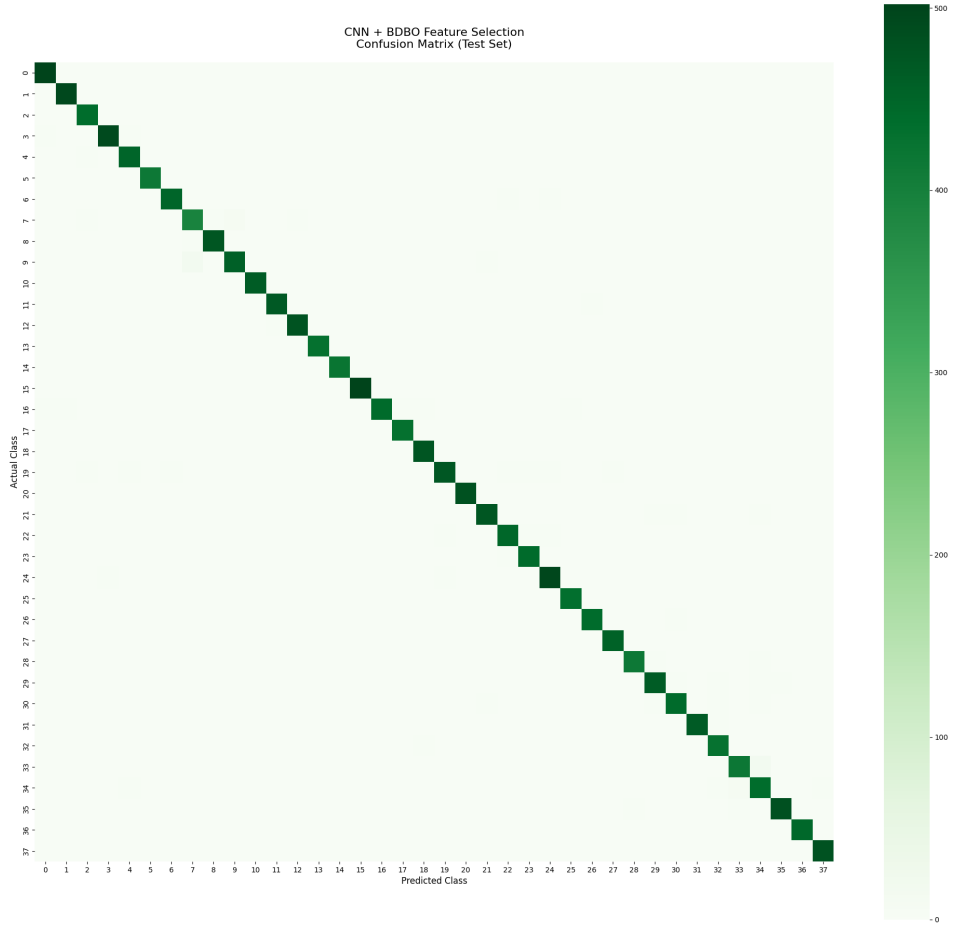


Figure 5.4: Confusion matrix of the CNN + BDBO head on the test set. Most mass lies on the diagonal.

5.7 Summary

The regularised CNN achieved 89.81% test accuracy end to end. Heads retrained on its pooled features reached about 98.5–98.7%, and the BDBO-selected 107-feature head achieved 98.47% accuracy and 0.9847 weighted F1-score with 58.20% fewer features than the full set. The matched comparison shows that feature selection preserved accuracy to within 0.2 percentage points but did not improve it, and that BPSO performed marginally better than BDBO in this run.

Chapter 6

Limitations, Threats to Validity, and Further Experiments

This chapter lists the limitations of the study, the factors that may affect the validity of the reported numbers, and the experiments needed to strengthen the conclusions.

6.1 Single Run and Absence of Significance Tests

Every number in Chapter 5 comes from one CNN training run and one run of each optimizer with one seed. Metaheuristics are stochastic, so different seeds may select different subsets of different sizes and give different accuracies. The gaps among the three heads (a few dozen test images) are small compared with the variation that could arise across seeds, and the notebook does not store per-image predictions from which a paired test such as McNemar's could be computed. The ranking of the three heads should therefore not be treated as established.

6.2 Surrogate Fitness

The wrapper fitness uses a k-NN classifier fitted on 6,000 samples, whereas the final classifier is a dense network trained on all 56,236 samples. A subset that is best for k-NN need not be best for the dense head, which may explain why the subset with the lower search fitness (BPSO) and the one with the higher test accuracy do not coincide in a simple way. Using the dense head itself, or a held-out portion distinct from the data used in the head's early stopping, as the fitness evaluator would give a more direct objective at a higher computational cost.

6.3 Validation Data Reuse

The wrapper fitness is computed on the validation feature set from the Keras split, and the CNN's own early stopping also used that split. The heads are trained on a fresh 80%/20% split of all 70,295 training-folder features, which means the head training set

includes images that were validation images for the CNN, and the CNN was trained on images that appear in the head's validation set. The test folder is not used at any stage of selection or training, so the final test numbers are not contaminated by this reuse, but the validation curves of the heads should not be interpreted as independent estimates.

6.4 Dataset Characteristics

The dataset is offline-augmented and its images are laboratory-style leaf photographs with simple backgrounds. If augmented variants of the same source leaf appear in both the training and `valid` folders, test accuracy would overestimate performance on new leaves; the notebook does not provide a way to check this. Results on field images with complex backgrounds, varied lighting, and multiple diseases per leaf are likely to be lower [1, 4]. No such images were tested.

6.5 End-to-End CNN Behaviour

The end-to-end CNN showed unstable validation accuracy (Figure 5.1) and an evaluation-mode accuracy on the training set (90.35%) well below the training-mode accuracy at the same epoch (96.60%). The cause was not investigated. Possible contributors include the interaction of augmentation, batch-normalization statistics, and mixed-precision arithmetic, but these are hypotheses only. Because the heads are trained on features from the same network, the quality of the features, and not only the head, may be affected, and a better-behaved backbone could change the relative value of feature selection.

6.6 Simplified Optimizer

The BDBO used here is a simplified adaptation of the published algorithm (Section 3.7), and its parameters were not tuned. BPSO reached a lower search fitness than BDBO in the single run reported, so the results do not demonstrate an advantage for BDBO over BPSO, nor do they show the converse.

6.7 Efficiency Claims

Feature selection is applied after the convolutional backbone, so the convolutional computation per image is unchanged; only the head becomes smaller (about 50%

fewer parameters for the BDBO head). Inference time, memory use, and energy were not measured, and no speed-up claim is supported by the recorded outputs.

6.8 Code Reproducibility Issues

The cell that trains the three heads refers to `train_data` and `test_data` objects with a `.classes` attribute that are not created in any other cell of the notebook, which suggests that a data-generator definition was removed or run in an earlier session. The reported image counts (70,295 and 17,572) are consistent with the folder sizes, but the notebook should be corrected to define these objects so that it runs from top to bottom. In addition, the generic single-image function in Section 11 of the notebook does not divide pixel values by 255 whereas the models were trained on rescaled inputs, so the later inference routines, which do rescale, are the ones that should be used.

6.9 Ethical and Practical Considerations

A classifier of this kind should support, not replace, expert diagnosis. A wrong prediction could lead to unnecessary treatment or to a missed infection, and the model cannot recognise diseases or crops outside its 38 classes. A deployed system should show confidence values, handle out-of-distribution images (for example, by rejecting low-confidence predictions), and clearly state the crops and conditions it was trained on.

6.10 Recommended Further Experiments

1. Repeat CNN training, BDBO, and BPSO over at least five seeds and report mean and standard deviation of accuracy, subset size, and fitness.

2. Store test predictions and apply a paired significance test between the full-feature, BPSO, and BDBO heads.

3. Compare with a filter method (for example, mutual information), with L1-regularised or dropout-based selection, and with a head trained on a random subset of the same size as the BDBO subset.

4. Evaluate a stronger, stable backbone (for example, a pretrained MobileNetV2) with and without selection.

5. Use the dense head or a held-out split as the wrapper fitness, and vary α and β to trace the accuracy–size trade-off.

6. Check for near-duplicate source images across the training and test folders, and test on independently collected field images.

7. Measure inference latency and memory on the target device.

6.11 Summary

The results are consistent and well documented but come from a single seed on a benchmark that may be easier than real use. The conclusions in Chapter 7 are written with these limitations in mind.

Chapter 7

Conclusions and Future Directions

This chapter summarises the findings and identifies directions for future work.

7.1 Conclusions

This report built a regularised CNN for 38-class leaf-disease classification and used it as a feature extractor for a Binary Dung Beetle Optimizer wrapper that selects a subset of its 256 pooled features. On the 17,572-image held-out test folder, the end-to-end CNN reached 89.81% accuracy. Dense heads retrained on the frozen features reached 98.66% with all 256 features, 98.68% with 120 BPSO-selected features, and 98.47% with 107 BDBO-selected features. The BDBO subset has weighted precision, recall, and F1-score of 98.48%, 98.47%, and 98.47%, with the lowest per-class F1-scores (94.81%–95.86%) on Tomato Target Spot, Tomato Spider Mites, and Corn Cercospora Leaf Spot.

The main finding is a trade-off and not an accuracy gain: BDBO removed 58.20% of the features and the head's parameter count fell by about half, at a cost of 0.19 percentage points in test accuracy compared with the full-feature head. The improvement from 89.81% to 98.47% reflects retraining a classifier on the extracted features and should not be credited to feature selection. BPSO reached a slightly lower search fitness and a slightly higher test accuracy than BDBO in this single run, so the work does not demonstrate that BDBO outperforms BPSO.

The work also illustrates the importance of the baseline. Comparing a hybrid model against an unmatched baseline would have suggested an 8.66-point benefit from the optimizer, whereas the matched comparison shows a difference of less than one point in either direction. These conclusions are limited by the single seed, the surrogate fitness, and the benchmark nature of the dataset (Chapter 6).

7.2 Scope for Future Study

- Repeat the experiments across multiple seeds with significance testing, and tune the optimizer parameters and the weights α and β.

- Replace or complement the k-NN surrogate with the dense head as the fitness evaluator.
- Compare BDBO with other metaheuristics and with filter and embedded selection methods, and test whether the selected subsets transfer between CNN training runs.
- Investigate and fix the instability of the end-to-end CNN's validation accuracy, and evaluate pretrained backbones.
- Test the pipeline on field-captured images and add confidence-based rejection of out-of-scope inputs.
- Measure inference time and memory to determine whether the smaller head gives a practical benefit, and consider pruning the convolutional channels themselves if backbone cost is the bottleneck.
- Package the trained pipeline as a mobile or web application for growers, with expert review of its outputs.

References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, art. 1419, 2016, doi: 10.3389/fpls.2016.01419.

[2] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv:1511.08060, 2015.

[3] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018, doi: 10.1016/j.compag.2018.01.009.

[4] J. G. A. Barbedo, "Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification," *Computers and Electronics in Agriculture*, vol. 153, pp. 46–53, 2018, doi: 10.1016/j.compag.2018.08.013.

[5] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019, doi: 10.1016/j.compag.2018.03.032.

[6] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[8] M. Lin, Q. Chen, and S. Yan, "Network in network," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2014, arXiv:1312.4400.

[9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015, arXiv:1412.6980.

[10] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[11] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks*, vol. 4, 1995, pp. 1942–1948.

[12] J. Kennedy and R. C. Eberhart, "A discrete binary version of the particle swarm algorithm," in *Proc. IEEE Int. Conf. Systems, Man, and Cybernetics*, vol. 5, 1997, pp. 4104–4108.

[13] S. Mirjalili and A. Lewis, "S-shaped versus V-shaped transfer functions for binary particle swarm optimization," *Swarm and Evolutionary Computation*, vol. 9, pp. 1–14, 2013.

[14] J. Xue and B. Shen, "Dung beetle optimizer: A new meta-heuristic algorithm for global optimization," *The Journal of Supercomputing*, vol. 79, pp. 7305–7336, 2023, doi: 10.1007/s11227-022-04959-6.

[15] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] "New Plant Diseases Dataset," Kaggle dataset (accessed via `kagglehub` as `vipoooool/new-plant-diseases-dataset`). [Online]. Available: https://www.kaggle.com/datasets/vipoooool/new-plant-diseases-dataset